

Guía paso a paso: Tests Unitarios

JUnit 5 + Mockito — Proyecto API-scooter

Hola. En esta guía vamos a recorrer juntos, de principio a fin, cómo escribir tests unitarios para las tres capas del proyecto: el Service, el Controller y las validaciones del Model. Tómalo con calma y ve probando cada paso en tu propio editor; se aprende mucho mejor escribiendo el código que solo leyéndolo.

0. Conceptos rápidos antes de empezar

Antes de escribir una sola línea, quiero que tengas claras tres ideas. Si las entiendes, todo lo demás cae por su propio peso.

¿Qué es un test unitario? Es una prueba que examina **una sola clase de forma aislada**. Las dependencias de esa clase (por ejemplo, el repositorio) no se ejecutan de verdad: las reemplazamos por **mocks**, que son objetos falsos que tú programas para que respondan exactamente lo que necesitas. Gracias a esto puedes probar tu lógica sin levantar una base de datos.

JUnit 5 es el framework que ejecuta los tests. Estas son las piezas que usarás:

Anotación / método	Para qué sirve
@Test	Marca un método como test.
@BeforeEach	Código que corre antes de CADA test (para preparar datos).
@DisplayName("...")	Nombre legible del test en el reporte.
assertEquals(esperado, real)	Verifica que dos valores sean iguales.
assertTrue / assertFalse	Verifica una condición booleana.
assertNull / assertNotNull	Verifica si algo es nulo o no.
assertThrows(Exc.class, () -> ...)	Verifica que se lance una excepción.

Mockito crea y controla los mocks. Lo que usarás:

Elemento	Para qué sirve
@ExtendWith(MockitoExtension.class)	Activa Mockito en la clase de test.
@Mock	Crea un objeto falso de una dependencia.
@InjectMocks	Crea la clase real bajo prueba e inyecta los @Mock dentro de ella.
when(x).thenReturn(y)	"Cuando llamen a x, devuelve y".
when(x).thenThrow(e)	"Cuando llamen a x, lanza la excepción e".

Elemento	Para qué sirve
verify(mock).metodo()	Verifica que el método del mock fue llamado.
verify(mock, never())	Verifica que NO fue llamado.
any(), anyLong(), eq()	"Comodines" para los argumentos esperados.

El patrón AAA. Todo test bien escrito sigue tres pasos: **Arrange** (preparar los datos), **Act** (ejecutar el método que pruebas) y **Assert** (verificar el resultado). Cuando veas estos tres bloques en cada test, sabrás que vas por buen camino.

1. Preparar el pom.xml

Spring Boot ya empaqueta JUnit 5, Mockito y AssertJ dentro de un único starter llamado spring-boot-starter-test. Eso significa que no necesitas añadir esas librerías por separado. Asegúrate de tener **solo esta dependencia** de test en tu pom.xml (sin la etiqueta <version>, porque la gestiona el parent):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Con esa sola línea ya tienes disponibles junit-jupiter, mockito-core, mockito-junit-jupiter y assertj. No hace falta nada más.

2. Estructura de carpetas de los tests

Aquí empieza el trabajo de verdad. La regla es sencilla: los tests **espejan el paquete de la clase que prueban**, pero viven bajo src/test. Si tu clase está en Service, su test va en Service; si está en model, su test va en model. Así cualquiera encuentra el test de una clase sin pensarlo.

```
src/test/java/cl/duoc/API_scooter/
|-- Service/    ScooterServiceTest.java
|-- Controller/ ScooterControllerTest.java
`-- model/     ScooterModelValidationTest.java
```

Consejo: En src/test solo deben vivir clases que terminan en ...Test y que contienen métodos @Test. Nunca pongas ahí código de producción.

3. Test del Service (el caso clásico de Mockito)

El scooterService depende de scooterRepository. La idea es **simular el repositorio con un mock** y comprobar que la lógica del service hace lo correcto. Crea el archivo src/test/java/cl/duoc/API_scooter/Service/ScooterServiceTest.java:

```

package cl.duoc.API_scooter.Service;

import cl.duoc.API_scooter.Repository.scooterRepository;
import cl.duoc.API_scooter.model.scooterModel;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class) // 1. Activa Mockito
class ScooterServiceTest {

    @Mock // 2. Repositorio falso
    private scooterRepository repository;

    @InjectMocks // 3. Service real con el mock inyectado
    private scooterService service;

    private scooterModel scooter;

    @BeforeEach // 4. Datos frescos antes de cada test
    void setUp() {
        scooter = new scooterModel();
        scooter.setId(1L);
        scooter.setCodSe("SCOOT-STGO-001");
        scooter.setDispo("DISPONIBLE");
    }

    // ----- READ ALL -----
    @Test
    @DisplayName("getAllScooters devuelve la lista del repositorio")
    void getAllScooters_ok() {
        when(repository.findAll()).thenReturn(List.of(scooter)); // Arrange
        List<scooterModel> resultado = service.getAllScooters(); // Act
        assertEquals(1, resultado.size()); // Assert
        assertEquals("SCOOT-STGO-001", resultado.get(0).getCodSe());
        verify(repository).findAll();
    }

    // ----- READ BY ID -----
    @Test
    @DisplayName("getScooterById devuelve el scooter cuando existe")
    void getScooterById_existe() {
        when(repository.findById(1L)).thenReturn(Optional.of(scooter));
        Optional<scooterModel> resultado = service.getScooterById(1L);
        assertTrue(resultado.isPresent());
        assertEquals("SCOOT-STGO-001", resultado.get().getCodSe());
    }
}

```

```

@Test
@DisplayName("getScooterById devuelve vacio cuando no existe")
void getScooterById_noExiste() {
    when(repository.findById(99L)).thenReturn(Optional.empty());
    assertTrue(service.getScooterById(99L).isEmpty());
}

// ----- READ BY CODIGO DE SERIE -----
@Test
@DisplayName("getScooterByCodSe devuelve el scooter por codigo de serie")
void getScooterByCodSe_ok() {
    when(repository.findByCodSe("SCOOT-STGO-001")).thenReturn(Optional.of(scooter));
    Optional<scooterModel> resultado = service.getScooterByCodSe("SCOOT-STGO-001");
    assertTrue(resultado.isPresent());
}

// ----- CREATE -----
@Test
@DisplayName("createScooter guarda y devuelve el scooter")
void createScooter_ok() {
    when(repository.save(scooter)).thenReturn(scooter);
    scooterModel resultado = service.createScooter(scooter);
    assertNotNull(resultado);
    assertEquals("SCOOT-STGO-001", resultado.getCodSe());
    verify(repository).save(scooter);
}

// ----- UPDATE (PUT) -----
@Test
@DisplayName("updateScooter actualiza cuando el scooter existe")
void updateScooter_existe() {
    scooterModel detalles = new scooterModel();
    detalles.setCodSe("SCOOT-STGO-999");
    detalles.setDispo("MANTENIMIENTO");

    when(repository.findById(1L)).thenReturn(Optional.of(scooter));
    when(repository.save(any(scooterModel.class))).thenAnswer(inv -> inv.getArgument(0));

    scooterModel resultado = service.updateScooter(1L, detalles);

    assertNotNull(resultado);
    assertEquals("SCOOT-STGO-999", resultado.getCodSe());
    assertEquals("MANTENIMIENTO", resultado.getDispo());
    verify(repository).save(scooter);
}

@Test
@DisplayName("updateScooter devuelve null cuando el scooter NO existe")
void updateScooter_noExiste() {
    when(repository.findById(99L)).thenReturn(Optional.empty());
    scooterModel resultado = service.updateScooter(99L, new scooterModel());
    assertNull(resultado);
    verify(repository, never()).save(any()); // nunca debe guardar
}

// ----- PATCH (cambiar disponibilidad) -----

```

```

@Test
@DisplayName("updateScooterDispo cambia la disponibilidad cuando existe")
void updateScooterDispo_existe() {
    when(repository.findByCodSe("SCOOT-STGO-001")).thenReturn(Optional.of(scooter));
    when(repository.save(any(scooterModel.class))).thenAnswer(inv -> inv.getArgument(0));

    Optional<scooterModel> resultado =
        service.updateScooterDispo("SCOOT-STGO-001", "EN_USO");

    assertTrue(resultado.isPresent());
    assertEquals("EN_USO", resultado.get().getDispo());
}

@Test
@DisplayName("updateScooterDispo devuelve vacio cuando no existe")
void updateScooterDispo_noExiste() {
    when(repository.findByCodSe("NADA")).thenReturn(Optional.empty());
    assertTrue(service.updateScooterDispo("NADA", "EN_USO").isEmpty());
    verify(repository, never()).save(any());
}

// ----- DELETE BY ID -----
@Test
@DisplayName("deleteScooter elimina y devuelve true cuando existe")
void deleteScooter_existe() {
    when(repository.existsById(1L)).thenReturn(true);
    boolean resultado = service.deleteScooter(1L);
    assertTrue(resultado);
    verify(repository).deleteById(1L);
}

@Test
@DisplayName("deleteScooter devuelve false cuando NO existe")
void deleteScooter_noExiste() {
    when(repository.existsById(99L)).thenReturn(false);
    boolean resultado = service.deleteScooter(99L);
    assertFalse(resultado);
    verify(repository, never()).deleteById(anyLong());
}

// ----- DELETE BY CODIGO DE SERIE -----
@Test
@DisplayName("deleteScooterByCodSe elimina cuando existe")
void deleteScooterByCodSe_existe() {
    when(repository.findByCodSe("SCOOT-STGO-001")).thenReturn(Optional.of(scooter));
    boolean resultado = service.deleteScooterByCodSe("SCOOT-STGO-001");
    assertTrue(resultado);
    verify(repository).deleteByCodSe("SCOOT-STGO-001");
}

@Test
@DisplayName("deleteScooterByCodSe devuelve false cuando no existe")
void deleteScooterByCodSe_noExiste() {
    when(repository.findByCodSe("NADA")).thenReturn(Optional.empty());
    assertFalse(service.deleteScooterByCodSe("NADA"));
    verify(repository, never()).deleteByCodSe(anyString());
}
}

```

Las piezas clave, explicadas

- `@InjectMocks` construye el `scooterService` real y le mete dentro el `@Mock repository`. Por eso no necesitas escribir `new` a mano.
- `when(...).thenReturn(...)` programa la respuesta del mock. Si el service llama a un método del repositorio que no programaste, el mock devuelve `null` o una lista vacía por defecto.
- `thenAnswer(inv -> inv.getArgument(0))` hace que `save(x)` devuelva el mismo objeto `x` que recibió. Es útil cuando el service modifica el objeto y luego lo guarda.
- `verify(repository, never()).save(any())` comprueba que en el camino "no existe" nunca se guarda nada. Fíjate que aquí estás probando el **comportamiento**, no solo el valor de retorno.

4. Test del Controller (Mockito puro, sin levantar Spring)

El `scooterController` depende de `scooterService`. Lo mockeamos, llamamos a los métodos del controller directamente y verificamos el `ResponseEntity` (es decir, el código HTTP y el cuerpo de la respuesta). Es muy rápido porque no necesita arrancar un servidor. Crea `src/test/java/cl/duoc/API_scooter/Controller/ScooterControllerTest.java`:

```
package cl.duoc.API_scooter.Controller;

import cl.duoc.API_scooter.Service.scooterService;
import cl.duoc.API_scooter.model.scooterModel;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

import java.util.List;
import java.util.Map;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class ScooterControllerTest {

    @Mock
    private scooterService service;

    @InjectMocks
    private scooterController controller;

    private scooterModel scooter;
```

```

@BeforeEach
void setUp() {
    scooter = new scooterModel();
    scooter.setId(1L);
    scooter.setCodSe("SCOOT-STGO-001");
    scooter.setDispo("DISPONIBLE");
}

@Test
@DisplayName("obtenerTodos responde 200 con la lista")
void obtenerTodos_ok() {
    when(service.getAllScooters()).thenReturn(List.of(scooter));
    ResponseEntity<List<scooterModel>> resp = controller.obtenerTodos();
    assertEquals(HttpStatus.OK, resp.getStatusCode());
    assertEquals(1, resp.getBody().size());
}

@Test
@DisplayName("obtenerScooterPorCodigo responde 200 cuando existe")
void obtenerPorCodigo_existe() {
    when(service.getScooterByCodSe("SCOOT-STGO-001")).thenReturn(Optional.of(scooter));
    ResponseEntity<scooterModel> resp =
        controller.obtenerScooterPorCodigo("SCOOT-STGO-001");
    assertEquals(HttpStatus.OK, resp.getStatusCode());
    assertEquals("SCOOT-STGO-001", resp.getBody().getCodSe());
}

@Test
@DisplayName("obtenerScooterPorCodigo responde 404 cuando no existe")
void obtenerPorCodigo_noExiste() {
    when(service.getScooterByCodSe("NADA")).thenReturn(Optional.empty());
    ResponseEntity<scooterModel> resp = controller.obtenerScooterPorCodigo("NADA");
    assertEquals(HttpStatus.NOT_FOUND, resp.getStatusCode());
}

@Test
@DisplayName("obtenerScooterPorId responde 200 cuando existe")
void obtenerPorId_existe() {
    when(service.getScooterById(1L)).thenReturn(Optional.of(scooter));
    ResponseEntity<scooterModel> resp = controller.obtenerScooterPorId(1L);
    assertEquals(HttpStatus.OK, resp.getStatusCode());
}

@Test
@DisplayName("obtenerScooterPorId responde 404 cuando no existe")
void obtenerPorId_noExiste() {
    when(service.getScooterById(99L)).thenReturn(Optional.empty());
    ResponseEntity<scooterModel> resp = controller.obtenerScooterPorId(99L);
    assertEquals(HttpStatus.NOT_FOUND, resp.getStatusCode());
}

@Test
@DisplayName("crearScooter responde 201 CREATED")
void crearScooter_ok() {
    when(service.createScooter(scooter)).thenReturn(scooter);
    ResponseEntity<scooterModel> resp = controller.crearScooter(scooter);
    assertEquals(HttpStatus.CREATED, resp.getStatusCode());
    assertEquals("SCOOT-STGO-001", resp.getBody().getCodSe());
}

```

```

}

@Test
@DisplayName("actualizarDispoScooter responde 400 si falta el campo dispo")
void actualizarDispo_sinCampo() {
    ResponseEntity<scooterModel> resp =
        controller.actualizarDispoScooter("SCOOT-STGO-001", Map.of());
    assertEquals(HttpStatus.BAD_REQUEST, resp.getStatusCode());
    verify(service, never()).updateScooterDispo(anyString(), anyString());
}

@Test
@DisplayName("actualizarDispoScooter responde 200 cuando actualiza")
void actualizarDispo_ok() {
    scooter.setDispo("EN_USO");
    when(service.updateScooterDispo("SCOOT-STGO-001", "EN_USO"))
        .thenReturn(Optional.of(scooter));
    ResponseEntity<scooterModel> resp =
        controller.actualizarDispoScooter("SCOOT-STGO-001", Map.of("dispo", "EN_USO"));
    assertEquals(HttpStatus.OK, resp.getStatusCode());
    assertEquals("EN_USO", resp.getBody().getDispo());
}

@Test
@DisplayName("actualizarDispoScooter responde 404 cuando no existe")
void actualizarDispo_noExiste() {
    when(service.updateScooterDispo("NADA", "EN_USO")).thenReturn(Optional.empty());
    ResponseEntity<scooterModel> resp =
        controller.actualizarDispoScooter("NADA", Map.of("dispo", "EN_USO"));
    assertEquals(HttpStatus.NOT_FOUND, resp.getStatusCode());
}

@Test
@DisplayName("eliminarScooterPorId responde 204 cuando elimina")
void eliminarPorId_ok() {
    when(service.deleteScooter(1L)).thenReturn(true);
    ResponseEntity<Void> resp = controller.eliminarScooterPorId(1L);
    assertEquals(HttpStatus.NO_CONTENT, resp.getStatusCode());
}

@Test
@DisplayName("eliminarScooterPorId responde 404 cuando no existe")
void eliminarPorId_noExiste() {
    when(service.deleteScooter(99L)).thenReturn(false);
    ResponseEntity<Void> resp = controller.eliminarScooterPorId(99L);
    assertEquals(HttpStatus.NOT_FOUND, resp.getStatusCode());
}

@Test
@DisplayName("eliminarScooterPorCodigo responde 204 cuando elimina")
void eliminarPorCodigo_ok() {
    when(service.deleteScooterByCodSe("SCOOT-STGO-001")).thenReturn(true);
    ResponseEntity<Void> resp = controller.eliminarScooterPorCodigo("SCOOT-STGO-001");
    assertEquals(HttpStatus.NO_CONTENT, resp.getStatusCode());
}

@Test
@DisplayName("eliminarScooterPorCodigo responde 404 cuando no existe")

```



```

void eliminarPorCodigo_noExiste() {
    when(service.deleteScooterByCodSe("NADA")).thenReturn(false);
    ResponseEntity<Void> resp = controller.eliminarScooterPorCodigo("NADA");
    assertEquals(HttpStatus.NOT_FOUND, resp.getStatusCode());
}
}

```

Nota: Este enfoque prueba la **lógica del controller** (qué código HTTP devuelve según lo que retorna el service). No prueba el ruteo de URLs ni la deserialización del JSON; para eso existe `@WebMvcTest` con `MockMvc`, que ya es un test de integración de la capa web. Para "JUnit + Mockito puro", lo de arriba es lo correcto.

5. Test de validaciones del Model

El `scooterModel` tiene reglas de validación (`@NotBlank`, `@Pattern`). Estas se prueban con el **Validator de Jakarta Bean Validation**, sin Mockito: aquí no hay dependencias que simular, simplemente validamos el objeto y comprobamos si genera violaciones. Crea `src/test/java/cl/duoc/API_scooter/model/ScooterModelValidationTest.java`:

```

package cl.duoc.API_scooter.model;

import jakarta.validation.ConstraintViolation;
import jakarta.validation.Validation;
import jakarta.validation.Validator;
import jakarta.validation.ValidatorFactory;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class ScooterModelValidationTest {

    private static Validator validator;

    @BeforeAll
    static void initValidator() {
        ValidatorFactory factory = Validation.buildDefaultValidatorFactory();
        validator = factory.getValidator();
    }

    @Test
    @DisplayName("Un scooter valido no genera violaciones")
    void scooterValido() {
        scooterModel s = new scooterModel();
        s.setCodSe("SCOOT-STGO-001");
        s.setDispo("DISPONIBLE");

        Set<ConstraintViolation<scooterModel>> v = validator.validate(s);
        assertTrue(v.isEmpty());
    }
}

```

```

}

@Test
@DisplayName("codSe en blanco genera violacion (@NotBlank)")
void codSeEnBlanco() {
    scooterModel s = new scooterModel();
    s.setCodSe(""); // invalido
    s.setDispo("DISPONIBLE");

    Set<ConstraintViolation<scooterModel>> v = validator.validate(s);
    assertFalse(v.isEmpty());
}

@Test
@DisplayName("dispo con valor no permitido genera violacion (@Pattern)")
void dispoInvalido() {
    scooterModel s = new scooterModel();
    s.setCodSe("SCOOT-STGO-001");
    s.setDispo("VOLANDO"); // no es DISPONIBLE | EN_USO | MANTENIMIENTO

    Set<ConstraintViolation<scooterModel>> v = validator.validate(s);
    assertFalse(v.isEmpty());
}

@Test
@DisplayName("Los tres estados permitidos son validos")
void estadosPermitidos() {
    for (String estado : new String[]{"DISPONIBLE", "EN_USO", "MANTENIMIENTO"}) {
        scooterModel s = new scooterModel();
        s.setCodSe("SCOOT-STGO-001");
        s.setDispo(estado);
        assertTrue(validator.validate(s).isEmpty(), "Deberia ser valido: " + estado);
    }
}
}

```

6. Ejecutar los tests

Tienes dos formas de lanzar los tests. Te recomiendo conocer ambas: la de IntelliJ es cómoda mientras escribes, y la de comandos es la que usarás en integración continua o cuando trabajes desde la terminal.

Opción A: desde IntelliJ IDEA

- **Un solo método:** abre la clase de test y pulsa la flecha verde (▶) que aparece en el margen izquierdo, justo al lado del nombre del método. Elige "Run".
- **Una clase completa:** pulsa la flecha verde (▶) que está al lado del nombre de la clase. Ejecutará todos sus métodos @Test.
- **Todo el proyecto:** haz clic derecho sobre la carpeta src/test/java en el panel Project y elige "Run 'All Tests'". También puedes usar el atajo Ctrl+Shift+F10 (Windows/Linux) o Ctrl+Shift+R (Mac) estando dentro de un test.

- **Ver resultados:** se abre la ventana "Run" abajo. Verde significa que pasó; rojo, que falló. Al hacer clic en un test rojo verás exactamente qué assert falló y en qué línea.

Consejo: Si IntelliJ no reconoce las anotaciones de test, recarga el proyecto Maven desde el panel "Maven" (icono de refrescar) para que descargue spring-boot-starter-test.

Opción B: desde la terminal (Maven)

Sitúate en la carpeta raíz del proyecto (API-scooter) y usa estos comandos:

```
# Ejecuta TODOS los tests
mvn test

# Ejecuta solo una clase
mvn -Dtest=ScooterServiceTest test

# Ejecuta solo un metodo
mvn -Dtest=ScooterServiceTest#getAllScooters_ok test
```

El reporte queda en target/surefire-reports/. Si todo está bien, al final verás algo como esto:

```
Tests run: 24, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Consejo: Por cada método del service, prueba al menos dos caminos: el **feliz** (existe / éxito) y el **alternativo** (no existe / error). Eso es exactamente lo que le da valor real a un test unitario.